



Rishat Maksudov

Software Engineer - C

CONTACT DETAILS

✉ orish.maks@gmail.com
☎ +99890 138 02 49
📍 Tashkent, Uzbekistan
📧 wildluck.dev
📁 github.com/wildluck

EDUCATION

Bachelor's Degree in Software Engineering
Innopolis University
2017 - 2021

Secondary Education (exact sciences)
Academic Lyceum under
Westminster International
University in Tashkent
2014 - 2017

SKILLS

C (C11 / C17) - Expert
Linux / POSIX - Expert
Networking stack - Expert
eBPF / NFQ - Advanced
DPDK / io_uring - Advanced
NVMe / NVMe-oF - Advanced
C++ - Advanced
Python - Advanced
Bash / Shell - Advanced

TOOLS

GDB, Valgrind, perf / FlameGraph
tcpdump, Wireshark
Make, CMake
pytest, GoogleTest
systemd, RPM / Spec
Git, Docker

LANGUAGES

Russian - Native
English - Advanced

Summary

Systems C programmer focused on networking and low-level Linux internals. 4+ years of production C across router firmware, NAT subsystems, and network virtualization. Comfortable with kernel-adjacent tooling - eBPF, NFQ, io_uring - and across the day-to-day of systems work: debugging, profiling, packaging, and operational glue. Available for work.

Work Experience

Software Engineer, Wellstor

January 2025 - Present

Work in C on the disk subsystem of a distributed block storage product, contributing inside the SPDK codebase.

- Currently building Prometheus metrics integration for the SPDK service - runtime observability over the disk subsystem.
- Added custom SPDK commands and RPCs for disk management - volume splits, RAID creation/management, and related operations.
- Designed and implemented a disk heartbeat mechanism across nodes on the multi-node test stand, used to detect disk-level connectivity issues.
- Added on-disk metadata for disks and RAID volumes, used to recover and reconfigure previously-added disks from their stored metadata on restart.
- Added automatic versioning for individual SPDK modules, so binary compatibility issues can be detected at runtime.
- Maintained the SPDK service: RPM .spec packaging and systemd units.
- Fixed a lock-up in SPDK's NVMe connection path that occurred when reaching a disk that had become unreachable over the network.
- Diagnosed and fixed several memory leaks across the disk subsystem using AddressSanitizer.

Software Engineer, Huawei

July 2022 - November 2024

Continued on the same firmware codebase; scope expanded to the NAT subsystem and the eNSP Pro Lite network simulator.

- Led the local NAT effort on the router with a small team of two, coordinating with a remote colleague at Huawei China; added the full CLI control surface for NAT configuration, refactored and extended ALG handling in legacy code for FTP, PPTP, RTSP and other protocols, and added new ALGs where needed.
- Extended eNSP Pro Lite (enterprise network simulation platform) from its initial device set to cover our AP, SOHO router, IoT devices, and mobile stations.
- Implemented virtual wireless connectivity inside the simulator using NFQ packet redirection and eBPF; added a link / physical state protocol for virtual connections.
- Simulated wireless environment effects - distance-based signal strength, packet loss, link-state changes - and station roaming, so mobile devices always associate with the strongest available AP.
- Built ubridge (a variation of ebridge) for joining interfaces across virtual devices into a single bridge, allowing eNSP Pro Lite to interoperate with external network simulators.
- Partnered with DevOps to build a pytest + scapy system-test framework: drive a QEMU-hosted SOHO via configuration fixtures, then validate NAT and IP routing behaviour against captured traffic (scapy, Wireshark).

Junior Software Engineer, Huawei

July 2021 - July 2022

Worked on firmware in C for AP, IoT, and SOHO router devices, focused on the CLI / control plane.

- Implemented IP address configuration and dynamic enable/disable of IP routes on address changes; added handling for link and physical/logical addresses.
- Built an ARP table with binary-tree fast lookup over an auto-expanding storage layer.
- Wrote GoogleTest unit tests covering the ARP table and surrounding control-plane code — lookup correctness, growth behaviour, edge cases.

Projects

living_dnd — github.com/wildluck/living_dnd · 2026 · C++23

Persistent D&D simulator: procedural hex terrain, dynamic weather, kingdoms, trade, and D&D 5e mechanical effects.

cppfolio — github.com/wildluck/cppfolio - wildluck.dev · 2026 · C++23, Crow, Python

Zero-JS portfolio server in C++23 with Crow; HTML templates and personal data baked into the binary at compile time via a Python codegen step.

minibroker — github.com/wildluck/minibroker · 2026 · C++

Lightweight pub/sub message broker: epoll-driven event loop, binary wire protocol, CRC32 integrity checks.

c_chat — github.com/wildluck/c_chat · 2026 · C

Multi-client TCP chat server in C: epoll reactor, thread pool, SHA-256 password authentication.